Laporan Tugas Besar

IF2224 Teori Bahasa Formal dan Otomata

Milestone 1

Semester II / 2025–2026

Disusun oleh:

GTFOTBFO (HBB)

Nathanael Imandatua Manurung - 13524021
Nicholas Wise Saragih Sumbayak - 13524037
Kurt Mikhael Purba - 13524065
Rava Khoman Tuah Saragih - 13524107

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **2**

2 Dasar Teori **2**
2.1 Pendefinisian Bahasa Pemrograman . . . 2
2.1.1 Hierarki Bahasa Pemrograman . . . 3
2.1.2 *Compiler* dan *Interpreter* . . . 3
2.2 Teori Kompilator (*Compiler Theory*) . . . 3
2.2.1 Analisis Leksikal (*Lexical Analysis*) . . . 4
2.3 Teori Bahasa Formal dan Otomata . . . 5
2.3.1 *Regular Expression* . . . 5
2.3.2 *Finite Automata* (FA) . . . 5

3 Perancangan & Implementasi **6**
3.1 Diagram Transisi DFA . . . 6
3.2 Implementasi Program (Lexical Analyzer) . . . 6
3.2.1 Struktur Proyek . . . 6
3.2.2 Penjelasan Modul/Kelas yang Dibentuk . . . 6
3.2.3 Klasifikasi Token pada Implementasi Scanner . . . 7

4 Experiments and Results **8**
4.1 Test Case Category A . . . 8
4.1.1 Test Case 1: *[Description]* . . . 8
4.1.2 Test Case 2: *[Description]* . . . 9
4.2 Test Case Category B . . . 9
4.3 Overall Analysis . . . 9
4.3.1 Algorithm A — Strengths and Limitations . . . 9
4.3.2 Algorithm B — Strengths and Limitations . . . 9

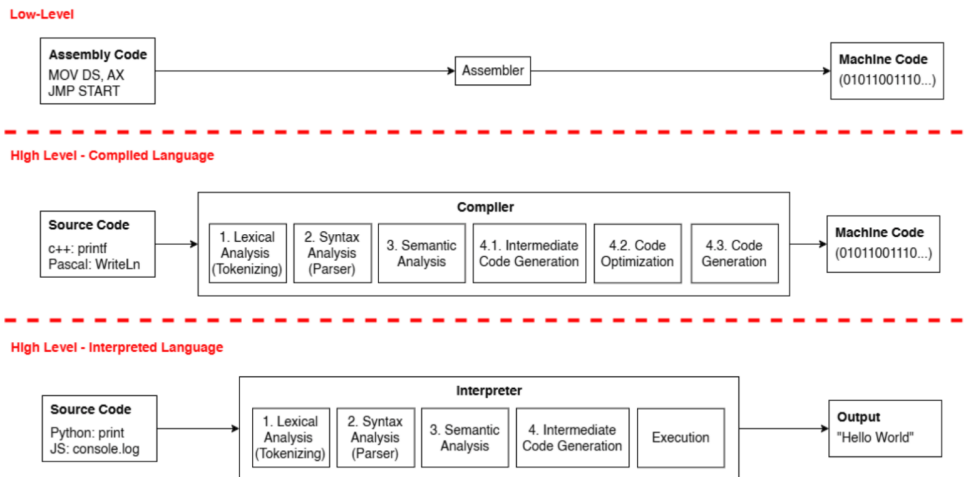
5 Conclusion **9**
5.1 Summary . . . 9
5.2 Suggestions for Future Work . . . 9
5.3 Individual Reflections . . . 9
5.3.1 Nathanael Imandatua Manurung - 13524021 . . . 9
5.3.2 Nicholas Wise Saragih Sumbayak - 13524037 . . . 10
5.3.3 Kurt Mikhael Purba - 13524065 . . . 10

References **11**

Appendix **12**

1 Pendahuluan

Bahasa Pemrograman diciptakan sebagai perantara komunikasi antara manusia yang mengerti logika dan bahasa tingkat tinggi dengan komputer yang hanya dapat mengerti bahasa mesin (*machine code*). Tanpa adanya bahasa pemrograman, manusia harus menuliskan instruksi komputer secara tingkat rendah kepada komputer yang hanya mengerti 0 dan 1. Dalam hierarkinya, bahasa tingkat tinggi (*high-level*) dirancang agar lebih mendekati bahasa manusia, namun hal ini menuntut proses transformasi yang berlapis agar dapat dipahami oleh CPU.



Gambar 1: Tipe-tipe bahasa pemrograman

Sumber: Asisten IF2224

Proses transformasi ini dilakukan melalui mekanisme *compiler* atau *interpreter*. Tahap awal dan fundamental dalam mekanisme tersebut adalah Analisis Leksikal (*Lexical Analysis*). Pada tahap ini, kode sumber yang berupa rangkaian karakter mentah dibaca dan dikelompokkan menjadi satuan makna terkecil yang disebut token, seperti kata kunci (*keyword*), pengenal (*identifier*), operator, dan konstanta. Analisis leksikal berperan krusial dalam menyederhanakan tugas tahap berikutnya, yaitu parsing, dengan membuang elemen yang tidak relevan seperti spasi dan komentar, serta mendeteksi simbol invalid sejak awal.

2 Dasar Teori

2.1 Pendefinisian Bahasa Pemrograman

Bahasa Pemrograman adalah cara manusia untuk dapat memberikan perintah (*instruction*) kepada sebuah komputer. Setiap bahasa pemrograman memiliki kata kunci dan aturannya masing-masing serta makna dari kata-kata itu yang disebut sintaks(*syntax*) dan semantik (*semantics*). Input yang terdapat kesalahan pada tingkat sintaks berarti input tersebut tidak valid untuk bahasa pemrograman tersebut, sedangkan kesalahan pada tingkat semantik umumnya berbentuk kesalahan logika.

2.1.1 Hierarki Bahasa Pemrograman

Berdasarkan tingkat abstraksinya, bahasa pemrograman diklasifikasikan menjadi dua kategori utama:

- ***Low-Level Language* (Bahasa Tingkat Rendah)**: Bahasa yang instruksinya mendekati arsitektur perangkat keras dan bahasa mesin (biner 0 dan 1). Contohnya adalah *Assembly*, yang memerlukan *Assembler* untuk menerjemahkan kode menjadi instruksi yang dapat dipahami CPU.
- ***High-Level Language* (Bahasa Tingkat Tinggi)**: Bahasa yang dirancang agar lebih mudah dibaca dan dipahami oleh manusia karena menggunakan kosakata yang menyerupai bahasa natural. Semakin tinggi tingkat bahasa tersebut, semakin banyak tahapan pemrosesan yang diperlukan untuk mengubahnya menjadi kode mesin. Contoh bahasa tingkat tinggi antara lain Pascal, C++, Python, dan Java.

2.1.2 *Compiler* dan *Interpreter*

Untuk menjalankan kode sumber yang ditulis dalam bahasa tingkat tinggi, diperlukan sebuah perangkat lunak penerjemah yang mengubah kode tersebut menjadi format yang dapat dieksekusi oleh mesin. Secara umum, terdapat dua mekanisme utama untuk mengubah bahasa tingkat tinggi ke bahasa mesin:

1. ***Compiler* (Kompilator)**: *Compiler* adalah program yang menerjemahkan seluruh *source code* secara utuh menjadi kode mesin (biasanya dalam bentuk *Object File*) sebelum program dijalankan. Proses kompilasi pada *compiler* umumnya terdiri dari empat tahapan utama: *Lexical Analysis*, *Syntax Analysis*, *Semantic Analysis*, dan *Intermediate Code Generation*. Karakteristik utama *compiler* adalah jika terdeteksi adanya kesalahan (*error*) di dalam *source code*, maka seluruh proses kompilasi akan gagal dan program tidak dapat dijalankan.
2. ***Interpreter***: Berbeda dengan *compiler*, *interpreter* memproses kode sumber baris demi baris secara langsung. *Interpreter* membaca, menerjemahkan, dan langsung mengeksekusi kode mesin tersebut baris demi baris. Hal ini memungkinkan program tetap berjalan normal hingga baris sebelum ditemukannya kesalahan; misalnya, jika terdapat *error* pada baris ke-10, baris 1 sampai 9 akan tetap berhasil dieksekusi. Meskipun mekanisme eksekusinya berbeda, *interpreter* sebenarnya memiliki tahapan pemrosesan internal yang serupa dengan *compiler*.

2.2 Teori Kompilator (*Compiler Theory*)

Proses kompilasi adalah urutan dari berbagai fase (*phases*) yang bekerja secara berurutan. Setiap fase mengambil input dari tahap sebelumnya, memprosesnya, dan memberikan output yang akan digunakan sebagai input untuk fase berikutnya. Secara umum, arsitektur sebuah kompilator terdiri dari fase-fase berikut:

1. ***Lexical Analysis***: Fase pertama yang bekerja sebagai pemindai teks (*text scanner*) pada kode sumber. Penjelasan lebih detail mengenai fase ini dibahas pada sub-bab tersendiri di bawah.

2. ***Syntax Analysis* (*Parsing*)**: Mengambil *token* yang dihasilkan oleh *lexical analysis* dan membangun pohon sintaksis (*parse tree*). Fase ini memeriksa kebenaran susunan *token* berdasarkan aturan tata bahasa (*grammar*) kode sumber.

3. ***Semantic Analysis***: Memeriksa apakah pohon sintaksis yang dibangun mengikuti aturan semantik bahasa, seperti kecocokan operasi tipe data dan memastikan *identifier* telah dideklarasikan sebelum digunakan. Fase ini menghasilkan keluaran berupa pohon sintaksis yang dianotasi.

4. ***Intermediate Code Generation***: Menghasilkan representasi kode perantara untuk mesin abstrak, yang posisinya menjembatani antara bahasa tingkat tinggi dan bahasa mesin.

5. ***Code Optimization***: Mengoptimalkan kode perantara dengan menghapus baris kode yang tidak perlu dan mengatur ulang urutan instruksi. Tujuannya adalah untuk mempercepat eksekusi program tanpa membuang sumber daya (*CPU*, memori).

6. ***Target Code Generation***: Merupakan fase sintesis akhir (*back-end*) yang menggunakan representasi kode perantara dan tabel simbol untuk menghasilkan program target atau bahasa mesin.

2.2.1 Analisis Leksikal (*Lexical Analysis*)

Sebagai fokus utama pada *milestone* ini, Analisis Leksikal adalah fase pertama dari sebuah kompilator. Fase ini memproses kode sumber yang ditulis dalam bentuk kalimat sintaksis dan memecahnya menjadi serangkaian unit bermakna yang disebut *token*.

Berdasarkan prinsip desain kompilator, mekanisme kerja dan tugas utama dari *lexical analyzer* meliputi:

- **Penyaringan Karakter**: *Lexical analyzer* membaca kode sumber sebagai aliran karakter (*character stream*) dan secara otomatis menghapus seluruh spasi kosong (*whitespace*) maupun komentar (*comments*) di dalam kode sumber.

- **Tokenisasi dan Pengenalan *Lexeme***: Barisan karakter (*alphanumeric*) yang memiliki makna diidentifikasi sebagai *lexeme*. *Lexeme* ini kemudian dicocokkan dengan aturan baku atau pola (*pattern*)—yang umumnya didefinisikan melalui *regular expression*—untuk divalidasi sebagai sebuah *token* (seperti kata kunci, konstanta, *identifier*, atau operator).

- **Kerja Sama dengan *Syntax Analyzer* (*Parser*)**: *Lexical analyzer* bekerja sangat erat dengan *syntax analyzer*. Ia memeriksa keberadaan *token* yang legal dan meneruskan data *token* tersebut kepada *syntax analyzer* hanya pada saat dibutuhkan atau diminta (*when it demands*).

- **Penanganan Kesalahan (*Error Handling*)**: Apabila *lexical analyzer* memindai urutan karakter yang tidak valid atau tidak dapat diklasifikasikan ke dalam pola *token* apa pun, maka ia akan langsung menghasilkan pesan kesalahan (*error*).

2.3 Teori Bahasa Formal dan Otomata

2.3.1 *Regular Expression*

Regular Expression (RE) adalah sebuah notasi matematis formal yang digunakan untuk mendeskripsikan suatu bahasa reguler. Dalam kompilator, RE digunakan secara luas untuk mendefinisikan *pattern* (pola) dari *token* secara akurat. Dengan menggunakan operasi dasar seperti *Concatenation* (penyambungan), *Union* (penggabungan alternatif), dan *Kleene Closure* (perulangan nol kali atau lebih), pembuat bahasa dapat mendefinisikan aturan rumit, misalnya: "Sebuah identifier harus diawali dengan huruf, diikuti oleh kombinasi huruf atau angka dengan jumlah berapa pun."

2.3.2 *Finite Automata* (FA)

Finite Automata (FA) atau *Finite State Machine* (FSM) adalah model komputasi matematika abstrak yang digunakan untuk mengenali *Regular Expression*. Dalam konteks kompilator, FA berfungsi sebagai mesin pemeriksa yang menelusuri karakter masukan untuk memvalidasi apakah urutan karakter tersebut (*lexeme*) membentuk *token* yang sah sesuai aturan bahasa.

Secara formal, sebuah *Finite Automata* didefinisikan secara matematis sebagai *5-tuple* $(Q, \Sigma, \delta, q_0, F)$, yang terdiri dari:

- Q : Himpunan hingga dari kedudukan atau *state*.
- Σ : Himpunan hingga dari simbol *input* atau alfabet (contoh: himpunan huruf, angka, dan simbol baca).
- δ : Fungsi transisi yang memetakan $Q \times \Sigma \rightarrow Q$ (menentukan *state* tujuan berdasarkan *state* saat ini dan simbol yang dibaca).
- q_0 : *State* awal (*initial state*), di mana $q_0 \in Q$. Mesin selalu memulai komputasi dari *state* ini.
- F : Himpunan *state* akhir atau penerima (*final/accepting state*), di mana $F \subseteq Q$.

Mesin FA membaca masukan berupa *string* karakter demi karakter secara sekuensial dari kiri ke kanan. Setiap kali membaca satu simbol, mesin akan berpindah dari *state* saat ini ke *state* berikutnya sesuai dengan aturan pada fungsi transisi (δ). Jika setelah seluruh karakter pada masukan selesai dibaca mesin berhenti pada salah satu *state* yang termasuk dalam himpunan F, maka masukan tersebut diterima (*accepted*) sebagai *token* yang valid. Sebaliknya, jika berhenti di luar F atau menemukan transisi yang tidak terdefinisi, maka terjadi *lexical error*.

Berdasarkan sifat penentuan fungsi transisinya, *Finite Automata* dibedakan menjadi dua jenis:

1. ***Nondeterministic Finite Automata* (NFA)**: Suatu mesin di mana sebuah *state* dapat memiliki nol, satu, atau lebih jalur transisi untuk satu simbol *input* yang sama. NFA juga mengizinkan perpindahan *state* tanpa perlu membaca karakter *input* (disebut transisi epsilon atau ϵ-transition).
2. ***Deterministic Finite Automata* (DFA)**: Suatu mesin di mana setiap *state* memiliki tepat satu jalur transisi secara mutlak untuk setiap simbol *input* di dalam Σ. Tidak ada ambiguitas jalur dan tidak ada transisi epsilon.

Dalam implementasi program *Lexical Analyzer* yang sebenarnya, model **DFA** adalah spesifikasi yang wajib digunakan. Hal ini karena sifat deterministiknya yang tidak ambigu memungkinkan aturan transisi dikonversi secara langsung menjadi algoritma terprogram menggunakan pendekatan *modular* dan penyeleksian kondisi, membaca satu demi satu karakter hingga menghasilkan klasifikasi *token* yang tepat.

3 Perancangan & Implementasi

3.1 Diagram Transisi DFA

3.2 Implementasi Program (Lexical Analyzer)

Tahapan pertama dalam *compiler* ini adalah *Lexical Analyzer* (Scanner) yang bertugas membaca *source code* karakter demi karakter dari `SourceBuffer` dan mengelompokkannya menjadi *Token* (seperti *identifier*, *keyword*, bilangan, dan operator). Implementasi pada `Scanner.cpp` dan `Scanner.hpp` dibangun dengan mereplikasi cara kerja *Deterministic Finite Automaton* (DFA) yang dirangkum pada diagram transisi DFA.

3.2.1 Struktur Proyek

Struktur implementasi untuk *lexical analyzer* pada proyek ini berfokus pada folder `src/lexical` dengan pembagian berkas sebagai berikut:

- `SourceBuffer.hpp` dan `SourceBuffer.cpp`: modul pembaca berkas sumber yang menyediakan akses karakter per karakter, termasuk utilitas *look-ahead*.
- `Scanner.hpp`: deklarasi kelas `Scanner` dan antarmuka fungsi pemindaian token.
- `Token.hpp`: representasi objek token beserta atributnya (jenis token, *lexeme*, dan metadata posisi jika ada).
- `TokenType.hpp`: definisi enumerasi semua terminal/token yang dikenali scanner.
- `Logger.hpp`: utilitas pelaporan pesan kesalahan/peringatan selama proses analisis leksikal.

3.2.2 Penjelasan Modul/Kelas yang Dibentuk

Implementasi bersifat modular agar setiap tanggung jawab terpisah dengan jelas.

- **Kelas `SourceBuffer`**: bertanggung jawab terhadap manajemen aliran karakter dari *source file*. Operasi inti seperti `peek()`, `advance()`, dan *matching* karakter digunakan scanner untuk merealisasikan transisi DFA.
- **Kelas `Scanner`**: inti mesin *lexical analyzer*. Kelas ini menjalankan simulasi DFA secara *ad-hoc* menggunakan struktur kontrol `switch-case`, `while`, dan `if-else`, lalu menghasilkan objek `Token` melalui fungsi pemindaian seperti `scanToken()`, `identifier()`, `number()`, dan `stringLiteral()`.
- **Kelas/Struktur `Token`**: wadah hasil klasifikasi *lexeme*. Setiap token merekam tipe terminal (dari `TokenType`) dan nilai *lexeme* yang sudah diekstrak dari aliran karakter.

- **`TokenType`**: kamus tipe token berbasis enumerasi untuk memastikan setiap hasil pemindaian memiliki kategori yang konsisten (misalnya `ident`, `intcon`, `realcon`, operator, delimiter, dan sebagainya).
- **`Logger`**: komponen pendukung untuk menampilkan *diagnostic message*, khususnya ketika scanner menemukan karakter ilegal atau komentar/string yang tidak tertutup.

3.2.3 Klasifikasi Token pada Implementasi Scanner

Ketika perulangan `scanToken()` dipanggil, program mensimulasikan dimulainya pemrosesan dari *Start State* (q_0). Bergantung pada karakter yang dibaca oleh `advance()`, cabang implementasi dibagi berdasarkan tipe token sebagai berikut:

- **Modul 1: Identifier & Keywords (q_1, q_2_ident)**
  Jika karakter awal adalah huruf atau *underscore* (diperiksa dengan `isAlpha()`), program akan memanggil fungsi `Scanner::identifier()`. Di sini, *state* membaca karakter selanjutnya selama karakter tersebut berupa alfanumerik (fungsi `isAlphaNumeric(peek())`), yang mensimulasikan perulangan (*looping*) pada *state* q_2_ident. Jika bertemu karakter non-alfanumerik, pointer diretract dan teks yang tersimpan dicocokkan dengan tabel *keyword* (`unordered_map`). Jika cocok, akan diubah menjadi *Token Keyword* (contoh: `programsy`, `beginsy`, `forsy`). Jika tidak, akan menjadi *Token* `ident`.
- **Modul 2: Numbers - intcon & realcon (q_4 hingga q_7)**
  Ketika menjumpai digit angka, program mendelegasikan pemrosesan ke `Scanner::number()`. Perulangan mewakilkan *state* q_4 dan q_5 yang murni membaca sekumpulan angka (`isdigit()`). Program kemudian melakukan *look-ahead* memeriksa apakah ada karakter titik '`.`' (q_6) yang segera diikuti oleh angka tambahan. Jika kondisi *match*, program mengakumulasi string menjadi tipe `double` untuk *Token* `realcon` (q_7). Jika hanya terdiri atas rentetan angka murni tanpa titik, pointer membaca mundur dan menjadi *Token* `intcon`.
- **Modul 3: Strings & Chars ($q_8, q_9_strchar$)**
  Kutip tunggal '`'`' memicu transisi pemanggilan ke `Scanner::stringLiteral()`. Program merepresentasikan *state* q_8 untuk mengevaluasi karakter apa pun di dalamnya secara beruntun. Saat divalidasi dan dicapai penutup '`'`', memori melacak panjang karakter yang terekam. Panjang berisi 1 menghasilkan terminal `charcon`, sisanya merujuk pada blok memori berupa `string`. Terdapat penanganan pada transisi *error* (q_error_3) jika deretan karakter mati dan tidak kunjung ditutup.
- **Modul 4: Operators & Delimiters**
  Karakter satuan seperti `+`, `-`, `*`, `[`, `]`, `,`, dan `;` secara ekuivalen diregisterkan menjadi simbol terminal mutlak karena merupakan terminal akhir / *accepting state*. Untuk tipe multi-operator seperti `>=`, `<=`, `:=`, kode menyebar jalur transisi lewat verifikasi `match()`:
  - Karakter `:` (q_{12}) akan mengecek `match('=')`. Jika ada, menjadi terminal operator `becomes` (`:=`). Jika tidak, terminal mengembalikan `colon` (`:`).
  - Karakter `<` (q_{10}) akan disiasati apakah mengevaluasi `=`, `>`, atau memicu *fallback* (merupakan `leq`, `neq`, atau `lss`).

  - Karakter `>` (q_{11}) memiliki fungsi serupa yang menunjuk preseden `>=` atau melakukuan pembacaan mundur sebagai `>`.
  - Karakter `=` (q_{55}) langsung menuju ke komputasi pembentukan leksik `eql` (konformitas yang ekuivalen dengan referensi diagram aslinya).

- **Modul 5: Comments (q_{14} hingga q_{17})**
  Saat sistem menemukan karakter `{` (q_{14}), iterasi berjalan membuang isi *buffer* karakter hingga bersentuhan langsung dengan simbol `}` yang merujuk logis pada transisi *state* $q_{15} \rightarrow$ comment1. Mekanisme setara terpakai untuk start `(` yang di-*lookahead* ke `*`, ia membiarkan perulangan maju hingga *encounter* elemen `*)` seperti pemblokiran rute transisi di $q_{16} \rightarrow q_{17} \rightarrow$ comment2. Jika instruksi berulang *End of File* tecapai, sistem menyuntikkan *alert error* yang relevan layaknya *termination pointer* pada q_error_1 atau q_error_2.

- **Modul 6: Whitespace**
  Peranan spasi seperti karakter utama *whitespace*, `\r`, serta `\t` lolos diabaikan karena memicu lompatan balik siklik kembali bereaksi identik dengan siklus statik *empty return* pada DFA, kecuali ganti baris (`\n`) yang dipakai menyimpan *tracking*.

Mesin secara natural membaca *source file* via `SourceBuffer` yang disalurkan melalui utilitas `peek()`, `advance()`, dan `match()` dan secara berkelanjutan menggunakan pergeseran pointer tersebut untuk merepresentasikan alur grafik DFA yang padu. Hasil kompilasi pergeseran *state operator* tersebut kemudian diangkut pada sub-rutin ekstraksi menggunakan penempatan variabel konstruktor kelas `Token`.

4 Experiments and Results

4.1 Test Case Category A

4.1.1 Test Case 1: *[Description]*

Input: Describe the input used.
Expected output: What the correct result should be.

[Screenshot placeholder]

Gambar 2: Test case 1 — result screenshot

Analysis: Explain whether the result is correct and why.

4.1.2 Test Case 2: *[Description]*

...

4.2 Test Case Category B

...

4.3 Overall Analysis

4.3.1 Algorithm A — Strengths and Limitations

Strengths:

- Strength one.
- Strength two.

Limitations:

- Limitation one.
- Limitation two.

4.3.2 Algorithm B — Strengths and Limitations

...

5 Conclusion

5.1 Summary

1. **Finding 1** — description.
2. **Finding 2** — description.
3. **Finding 3** — description.

5.2 Suggestions for Future Work

1. Idea for improvement or extension one.
2. Idea for improvement or extension two.

5.3 Individual Reflections

5.3.1 Nathanael Imandatua Manurung - 13524021

Role:
Reflection: Write your personal reflection here.

5.3.2 Nicholas Wise Saragih Sumbayak - 13524037

Role:
Reflection: ...

5.3.3 Kurt Mikhael Purba - 13524065

Role:
Reflection: ...

References

[1] A. Author, *Book Title*, Edition. City: Publisher, Year.

[2] A. Author and B. Author, "Article title," *Journal Name*, vol. X, no. Y, pp. 1–10, Month Year. https://doi.org/10.xxxx/xxxxx.

[3] Organisation, "Web resource title," Year. [Online]. Available: https://example.com.

Appendix

Resource	Link
GitHub Repository	https://github.com/your-org/your-repo
Live Deployment	https://your-deployment-url.example.com

Tabel 1: Repository and deployment links